Práctica 2: Algoritmos Divide y Vencerás

Néstor Eloy Gil Casado (Participación: 50 %)
Sergio Salvador Gil (Participación: 50 %)

8 de abril de 2026

Resumen

Memoria de la Práctica 2 de la asignatura de Algorítmica. Se abordan los problemas 2.2-Máximo y segundo máximo y 2.4-Inversiones en un vector mediante técnicas de Fuerza Bruta y Divide y Vencerás, analizando su eficiencia teórica y empírica, y discutiendo los resultados obtenidos.

Índice

Problema 2.2: Máximo y segundo máximo **3**

1. Algoritmo de fuerza bruta **3**
1.1. Descripción de la solución implementada 3
1.2. Lógica de la solución implementada (Caso de uso) 3
1.3. Ejecución del programa implementado 3
1.4. Análisis de la eficiencia teórica y empírica 4

2. Algoritmo divide y vencerás **4**
2.1. Descripción de la solución implementada 4
2.2. Lógica de la solución implementada (Caso de uso) 5
2.3. Ejecución del programa implementado (Cálculo del umbral) 6
2.4. Análisis de la eficiencia teórica y empírica 6

3. Discusión **6**

Problema 2.4: Inversiones en un vector **8**

4. Algoritmo de fuerza bruta **8**
4.1. Descripción de la solución implementada 8
4.2. Lógica de la solución implementada (Caso de uso) 8
4.3. Ejecución del programa implementado 8
4.4. Análisis de la eficiencia teórica y empírica 9

5. Algoritmo divide y vencerás **9**
5.1. Descripción de la solución implementada 9
5.2. Lógica de la solución implementada (Caso de uso) 10
5.3. Ejecución del programa implementado 11
5.4. Análisis de la eficiencia teórica y empírica 11

6. Discusión **11**

Problema 2.2: Máximo y segundo máximo

1. Algoritmo de fuerza bruta

1.1. Descripción de la solución implementada

Para resolver el problema mediante fuerza bruta con la mayor eficiencia posible, se ha implementado un algoritmo que recorre la matriz una única vez. Durante este recorrido, se mantienen actualizadas dos variables: el máximo absoluto (*max1*) y el segundo máximo estrictamente menor (*max2*). Si el elemento actual supera a *max1*, el antiguo máximo pasa a ser *max2* y se actualiza *max1*. Si no lo supera, pero es mayor que *max2* (y estrictamente menor que *max1*), se actualiza únicamente *max2*.

Algoritmo 1: FuerzaBrutaMaximos(M, n, m)

```
max1 ← −∞;
max2 ← −∞;
for i ← 0 to n-1 do
    for j ← 0 to m-1 do
        if M[i][j] > max1 then
            max2 ← max1;
            max1 ← M[i][j];
        end
        else if M[i][j] < max1 and M[i][j] > max2 then
            max2 ← M[i][j];
        end
    end
end
return max1, max2;
```

1.2. Lógica de la solución implementada (Caso de uso)

Supongamos una matriz M de 2×2:

$$M = \begin{pmatrix} 10 & 25 \\ 5 & 25 \end{pmatrix}$$

1. Inicializamos $max1 = -\infty$ y $max2 = -\infty$. 2. Leemos $M[0][0] = 10$. Como $10 > -\infty$, actualizamos $max1 = 10$. 3. Leemos $M[0][1] = 25$. Como $25 > 10$, actualizamos $max2 = 10$ y $max1 = 25$. 4. Leemos $M[1][0] = 5$. No es mayor que 25, pero tampoco es mayor que 10. Se ignora. 5. Leemos $M[1][1] = 25$. No es estrictamente mayor que 25, ni cumple ser estrictamente menor que $max1$ (25). Se ignora. Resultado final: $max1 = 25$, $max2 = 10$.

1.3. Ejecución del programa implementado

A continuación, se presenta la gráfica con los tiempos de ejecución empíricos del algoritmo de fuerza bruta comparados con la solución de Divide y Vencerás, variando el tamaño total de la matriz.

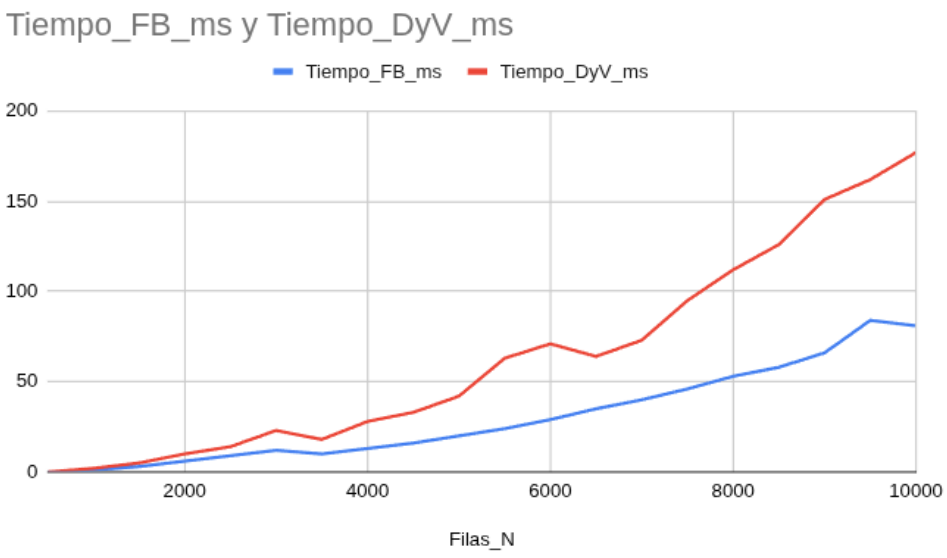

Figura 1: Tiempos de ejecución empíricos: Fuerza Bruta vs Divide y Vencerás.

1.4. Análisis de la eficiencia teórica y empírica

El algoritmo de fuerza bruta recorre todos los elementos de la matriz de tamaño $n \times m$. Las operaciones dentro de los bucles (comparaciones y asignaciones) tienen un coste constante $\Theta(1)$. Por tanto, el número total de operaciones es directamente proporcional al número de elementos. Sea $N = n \cdot m$ el tamaño total del problema, la eficiencia teórica es:

$$T(N) = \sum_{i=0}^{n-1} \sum_{j=0}^{m-1} \Theta(1) = \Theta(n \cdot m) = \Theta(N)$$

Este comportamiento lineal respecto al total de elementos se observa claramente en la gráfica empírica de la Figura 1, donde el tiempo de ejecución crece de forma proporcional al aumento del tamaño de la matriz.

2. Algoritmo divide y vencerás

2.1. Descripción de la solución implementada

Se ha implementado un algoritmo híbrido basado en la estrategia de Divide y Vencerás.

- **Dividir:** Si el número de elementos supera un umbral n_0, la matriz se divide en dos mitades (horizontal o verticalmente, dependiendo de qué dimensión sea mayor para mantener el balanceo).
- **Conquistar:** Se llama recursivamente a la función para cada mitad, devolviendo sus respectivos máximos. Si el tamaño es menor o igual al umbral n_0, se llama al algoritmo de fuerza bruta (caso base).
- **Combinar:** El $max1$ global es el máximo entre los dos $max1$ parciales. El $max2$ global se calcula enfrentando al "perdedor" de los $max1$ contra el $max2$ del cuadrante "ganador".

Algoritmo 2: MaximosDyV($v, f_{ini}, f_{fin}, c_{ini}, c_{fin}, n_0$)

```
/* Cálculo del tamaño del problema actual                                  */
num_filas ← f_fin − f_ini + 1;
num_cols ← c_fin − c_ini + 1;
n ← num_filas × num_cols;
/* Caso base: si el tamaño es menor al umbral                              */
if n ≤ n_0 then
    return ResolverDirectamente(v, f_ini, f_fin, c_ini, c_fin);
end
/* Fase 1: Dividir                                                         */
if num_filas ≥ num_cols then
    f_medio ← f_ini + ⌊num_filas/2⌋ − 1;
    /* Fase 2: Conquistar (Llamadas recursivas)                            */
    S1 ← MaximosDyV(v, f_ini, f_medio, c_ini, c_fin, n_0);
    S2 ← MaximosDyV(v, f_medio + 1, f_fin, c_ini, c_fin, n_0);
end
else
    c_medio ← c_ini + ⌊num_cols/2⌋ − 1;
    S1 ← MaximosDyV(v, f_ini, f_fin, c_ini, c_medio, n_0);
    S2 ← MaximosDyV(v, f_ini, f_fin, c_medio + 1, c_fin, n_0);
end
/* Fase 3: Combinar                                                        */
Solucion.max1 ← máx(S1.max1, S2.max1);
if S1.max1 > S2.max1 then
    Solucion.max2 ← máx(S1.max2, S2.max1);
end
else if S2.max1 > S1.max1 then
    Solucion.max2 ← máx(S2.max2, S1.max1);
end
else
    /* Si los máximos son iguales, comparamos sus segundos máximos */
    Solucion.max2 ← máx(S1.max2, S2.max2);
end
return Solucion;
```

2.2. Lógica de la solución implementada (Caso de uso)

Tomando la misma matriz M: 1. Se divide en dos mitades horizontales (filas 0 y 1). 2. Recursión Mitad 1 $[10, 25]$: Devuelve $max1 = 25, max2 = 10$. 3. Recursión Mitad 2 $[5, 25]$: Devuelve $max1 = 25, max2 = 5$. 4. Combinación: El $max1$ global es $\text{máx}(25, 25) = 25$. Como hay empate en los máximos absolutos, el $max2$ global será el mayor de los segundos máximos parciales: $\text{máx}(10, 5) = 10$. El resultado es idéntico a la fuerza bruta.

2.3. Ejecución del programa implementado (Cálculo del umbral)

Para determinar el umbral óptimo empírico (n_0), se ejecutó el algoritmo para una matriz estática de gran tamaño variando el valor del umbral.

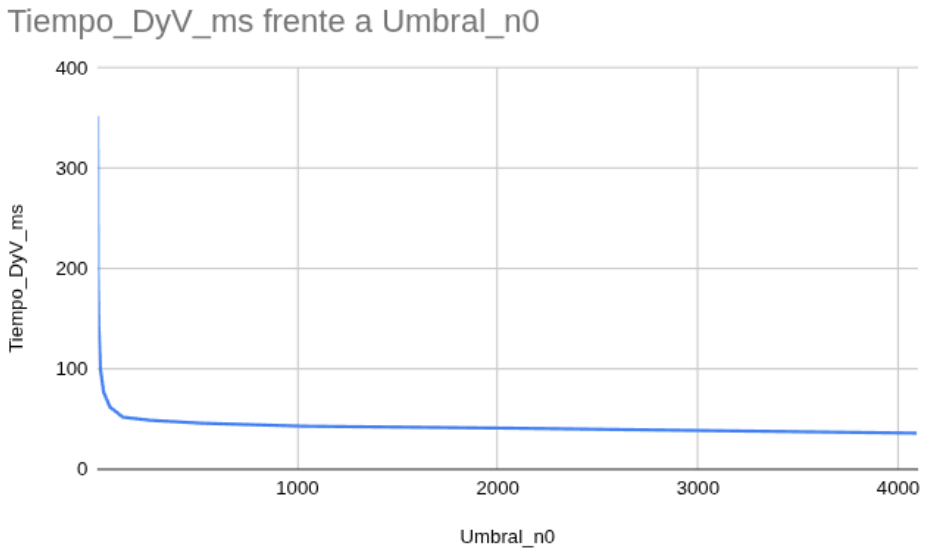

Figura 2: Tiempos de ejecución en función del umbral (n_0).

Como se observa en los datos recogidos, el tiempo disminuye a medida que el umbral aumenta. El tiempo más bajo se obtuvo para el umbral más alto probado (4096).

2.4. Análisis de la eficiencia teórica y empírica

Siguiendo el teorema maestro para resolver la ecuación de recurrencia, tenemos:

$$T(N) = \begin{cases} \Theta(1) & \text{si } N \leq n_0 \\ 2T(N/2) + \Theta(1) & \text{si } N > n_0 \end{cases}$$

Donde $k = 2$ (dos subproblemas), $b = 2$ (tamaño dividido a la mitad) y $a = 0$ (el coste de dividir y combinar es constante). Al aplicar el teorema maestro ($log_2 2 = 1 > 0$), la complejidad es:

$$T(N) \in \Theta(N^{log_2 2}) = \Theta(N)$$

Teóricamente, el coste es lineal respecto al número de elementos N, al igual que la fuerza bruta.

3. Discusión

Al analizar los resultados empíricos nos damos cuenta de un detalle importante: aunque los dos algoritmos (Fuerza Bruta y Divide y Vencerás) tienen la misma eficiencia teórica de $\Theta(N)$, usar la estrategia de Divide y Vencerás **no compensa para este problema en concreto**.

El motivo principal es que el paso de combinar las soluciones es muy simple (tiene un coste de $\Theta(1)$) y no nos aporta ninguna ventaja real de tiempo. Por el contrario, estar haciendo tantas llamadas recursivas añade un coste extra de memoria y de procesamiento que hace que el programa vaya más lento. Esto se ve muy claro en el estudio del umbral (Figura 2): notamos que cuanto más subimos el umbral (es decir, cuanto más trabajo le dejamos al algoritmo iterativo de fuerza bruta), más rápido termina el programa.

En conclusión, para buscar los valores máximos dentro de una matriz que no está ordenada, hacer un recorrido clásico con bucles (el método de fuerza bruta) resulta ser mucho más rápido y directo en la práctica.

Problema 2.4: Inversiones en un vector

4. Algoritmo de fuerza bruta

4.1. Descripción de la solución implementada

Para implementar el algoritmo mediante la fuerza bruta, hemos optado a realizar una función que recorre el vector mediante dos bucles anidados. El primer bucle itera en cada elemento y el segundo itera en los elementos con índices mayores (para evitar redundancia) al actual iterado, y comprueba si el elemento iterado por el segundo bucle es menor que el elemento iterado por el primero. Si eso es así, incrementa el contador de inversiones.

Algoritmo 3: CalcularInversionesFB(V, n)

```
n_inversiones ← 0;
for i ← 0 to n-1 do
    for j ← i+1 to n-1 do
        if V[i] > V[j] then
            n_inversiones ← n_inversiones + 1;
        end
    end
end
return n_inversiones;
```

4.2. Lógica de la solución implementada (Caso de uso)

Imaginemos el siguiente vector de 5 elementos:

$$v = (1, 2, 5, 4, 3)$$

Entonces, el algoritmo iteraría sin contar nada hasta el tercer elemento. Consiguientemente contaría dos inversiones (5 con 3 y 5 con 4), y luego en la siguiente iteración contaría una última inversión (4 con 3), y acabaría la función devolviendo que el vector tiene un máximo de 3 inversiones distintas.

4.3. Ejecución del programa implementado

A continuación, se presenta la gráfica con los tiempos de ejecución empíricos del algoritmo de fuerza bruta comparados con la solución de Divide y Vencerás, variando el tamaño total del vector, con elementos aleatorios.

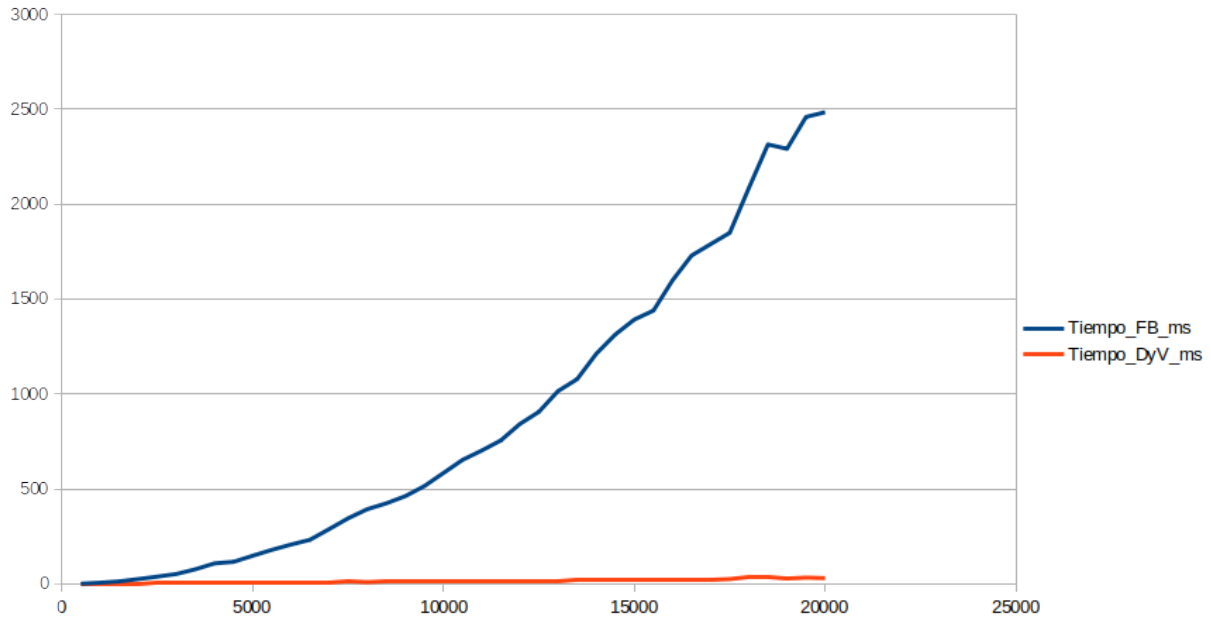

Figura 3: Tiempo entre fuerza bruta y DyV (ms)

4.4. Análisis de la eficiencia teórica y empírica

El algoritmo de fuerza bruta compara sistemáticamente todos los pares posibles $(v[i], v[j])$ del vector de tamaño n para verificar la condición de inversión ($i < j$ y $v[i] > v[j]$).

Para un vector de tamaño n, el número de comparaciones viene dado por el sumatorio de los elementos restantes en cada iteración del bucle externo. La eficiencia teórica se calcula como:

$$T(n) = \sum_{i=0}^{n-2} \sum_{j=i+1}^{n-1} \Theta(1) = \frac{(n-1) \cdot n}{2} = \Theta(n^2)$$

Este comportamiento cuadrático implica que, al duplicar el tamaño del vector, el tiempo de ejecución se cuadriplica aproximadamente. Esta tendencia es visible en la Figura 3, donde la curva de fuerza bruta muestra un crecimiento mucho más pronunciado que la alternativa de Divide y Vencerás.

5. Algoritmo divide y vencerás

5.1. Descripción de la solución implementada

La solución se basa en una modificación del algoritmo de ordenación *Merge Sort*. La estrategia consiste en:

- **Dividir:** El vector se divide recursivamente en dos mitades hasta llegar a subvectores de tamaño 1 o menores al umbral n_0.
- **Conquistar:** Se cuenta el número de inversiones en la mitad izquierda y en la mitad derecha de forma recursiva.

- **Combinar:** Se utiliza una función de mezcla (*merge*) que, además de ordenar los elementos, cuenta las ïnversiones cruzadas"(elementos en la mitad izquierda que son mayores que elementos en la mitad derecha).

Algoritmo 4: InversionesDyV(V, inicio, fin, n_0)

```
if fin - inicio + 1 ≤ n0 then
    return InversionesFB(V, inicio, fin);
end
mitad ← inicio + ⌊(fin − inicio)/2⌋;
inv ← InversionesDyV(V, inicio, mitad, n0);
inv ← inv + InversionesDyV(V, mitad + 1, fin, n0);
inv ← inv + MergeYContar(V, inicio, mitad, fin);
return inv;
```

Algoritmo 5: MergeYContar(V, inicio, mitad, fin)

```
i ← inicio, j ← mitad + 1, inv_c ← 0;
V_aux ← vector vacío;
while i ≤ mitad and j ≤ fin do
    if V[i] ≤ V[j] then
        V_aux.push(V[i]);
        i ← i + 1;
    end
    else
        V_aux.push(V[j]);
        /* Si V[i] >V[j], entonces todos los elementos desde i hasta
           la mitad forman una inversión con V[j]                    */
        inv_c ← inv_c + (mitad - i + 1);
        j ← j + 1;
    end
end
/* Añadir elementos restantes y volcar V_temp en V[inicio...fin]  */
return inv_c;
```

5.2. Lógica de la solución implementada (Caso de uso)

Sea el vector $v = (1, 2, 5, 4, 3)$:

1. Se divide en $v1 = (1, 2, 5)$ y $R = (4, 3)$.
2. Las llamadas recursivas devuelven las inversiones internas: $v1$ tiene 0 y $v2$ tiene 1 (par $(4, 3)$).
3. Durante la mezcla de $v1 = (1, 2, 5)$ y $v2 = (3, 4)$ (ya ordenados internamente):
   - Al comparar el 5 de $v1$ con el 3 de $v2$, como $5 > 3$, se contabilizan 1 inversión (y todas las que queden a la derecha del 5 en esa mitad, en este caso ninguna más).
   - Al comparar el 5 de $v1$ con el 4 de $v2$, como $5 > 4$, se contabiliza otra inversión.

4. Total: 0(izquierda) + 1(derecha) + 2(cruzadas) = 3 inversiones.

5.3. Ejecución del programa implementado

La gráfica de la figura 3 muestra que en la ejecución, el algoritmo de Divide y Vencerás es muchísimo más eficiente que el enfoqur basado en fuerza bruta. En comparación, el crecimiento del algoritmo de Divide y Vencerás no tarda más que unos milisegundos en resolver el problema incluso con vectores que cuentan con una gran cantidad de datos. En contraste, el tiempo de ejecución del algoritmo de fuerza bruta sale disparado, haciendo que con una cantidad alta de datos) el algoritmo de Divide y Vencerás tenga un crecimiento casi negligible en comparación.

5.4. Análisis de la eficiencia teórica y empírica

La eficiencia de este algoritmo sigue la estructura de los algoritmos de división balanceada. La ecuación de recurrencia es:

$$T(n) = \begin{cases} \Theta(1) & \text{si } n \leq n_0 \\ 2T(n/2) + \Theta(n) & \text{si } n > n_0 \end{cases}$$

Donde $2T(n/2)$ representa las dos llamadas recursivas y $\Theta(n)$ es el coste de la función de mezcla (*merge*) que recorre ambos subvectores una sola vez.

Aplicando el Teorema Maestro con $a = 2, b = 2$ y $f(n) = \Theta(n^1)$: 1. Calculamos $\log_b a = \log_2 2 = 1$. 2. Como $f(n) \in \Theta(n^1)$, estamos en el segundo caso del teorema.

$$T(n) = \Theta(n^{\log_2 2} \log n) = \Theta(n \log n)$$

La eficiencia $\Theta(n \log n)$ es significativamente superior a $\Theta(n^2)$, lo que permite procesar grandes volúmenes de datos con un crecimiento log-lineal.

6. Discusión

En este caso, a diferencia del problema 2.2, el algoritmo que produce la mejor eficiencia es el que utiliza el enfoque de Divide y Vencerás. Como este caso requiere comparar todos los elementos de un vector entre sí en vez de buscar un elemento dentro del mismo, el enfoque de fuerza bruta en este caso se dedica a hacer todas las combinaciones de comparaciones posibles, lo que se hace mediante dos bucles anidados, lo cual implica que la eficiencia del algortmo va a ser cuadrática, de $\Theta(n^2)$ lo que es fácilmente mejorable, y resulta no conveniente comparado con la eficiencia de $\Theta(n \log n)$ que da el algoritmo con el enfoque de Divide y Vencerás.